

# コンピュータ アーキテクチャ I 第8回

---

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

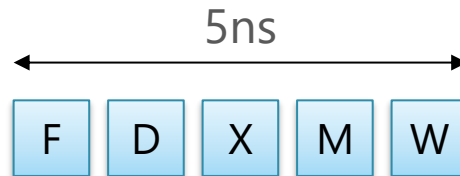
# 課題の解説

---

# 課題 7

## ■ 前提：

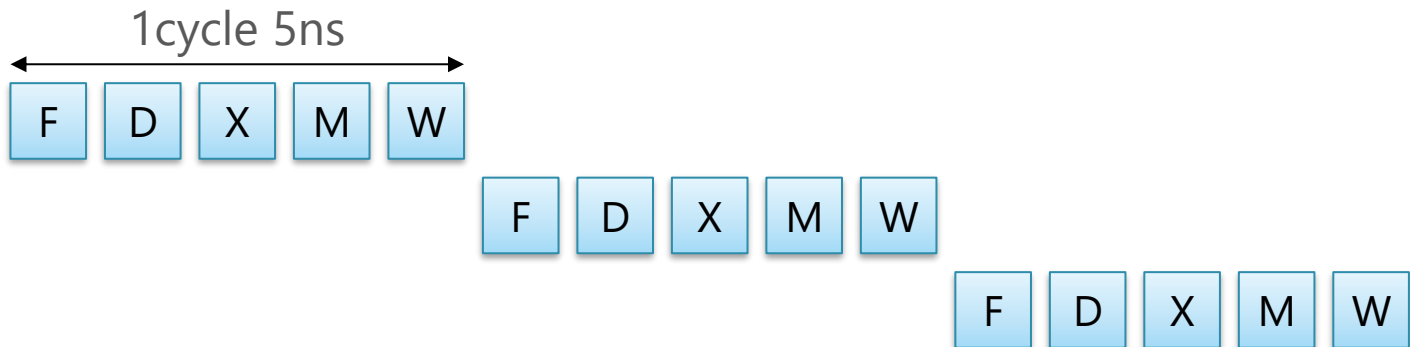
- 各命令は以下の5つの処理を経て実行されるものとする
  - F：フェッチ, D：デコード, X：演算,  
M：メモリアクセス, W：書き戻し
- これらの各処理には 1 nano second (ns) がかかるものとする
- 演算しか行わずメモリアクセスを伴わない命令でも, 必ず M を経るものとする
- フォワーディングは常に行うものとする
- 命令を実行するために「必要な時間」とは, 最初の命令のフェッチ開始から最後の命令の書き戻しが終わるまでの時間とする
  - たとえば以下の1命令を実行するために「必要な時間」は 5ns



# 課題 7

- (1) その場合の最大動作周波数はいくつになるか述べよ
  - $1 \text{ 秒} / 5\text{ns} = 200\text{MHz}$
  - 各命令の処理は 5ns かかり, それを 1 サイクルで処理するため
- (2) 以下の命令列を実行するのに必要な時間を計算せよ
  - $5 \text{ ns} \times 3 \text{ 命令} = 15\text{ns}$

```
sub x2←x3+x4  
add x1←x2+x3  
add x5←x6+x7
```

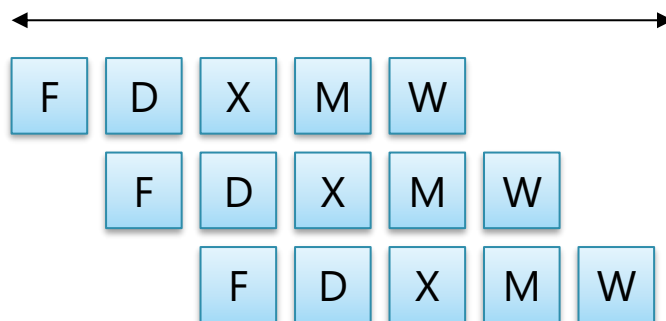


# 課題 7

- 前記の前提の下で, F,D,X,M,W の5ステージからなるパイプライン・プロセッサを構成することを考える  
ただし, in-order スカラ・プロセッサであるものとする
- (3) その場合の最大動作周波数はいくつになるか述べよ
  - 1 秒 / 1 ns = 1GHz
- (4) 以下の命令列を実行するのに必要な時間を計算せよ

sub x2 ← x3 + x4  
add x1 ← x2 + x3  
add x5 ← x6 + x7

- 7ns

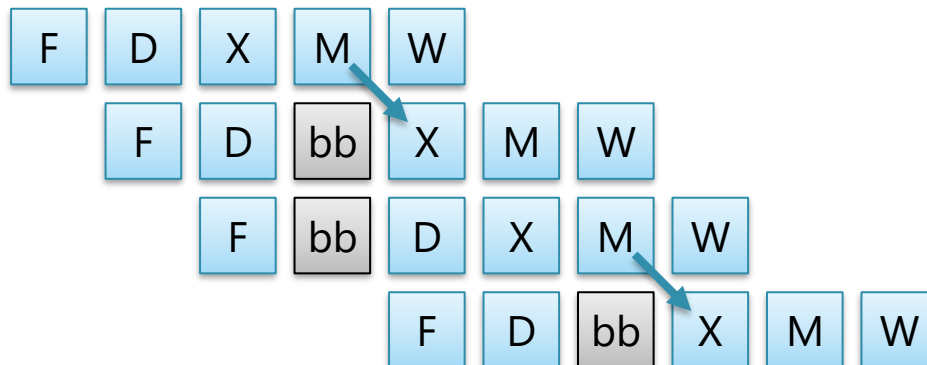


# 課題 7

- (5) 以下の命令列を実行するのに必要な時間を計算せよ  
依存関係のために必要な場合のみパイプラインを適宜ストールして実行するものとせよ

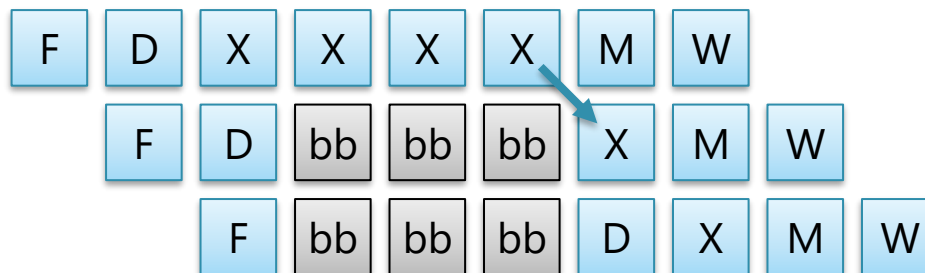
```
ld  x2 ← (x1)
add x1 ← x2 + x3
ld  x2 ← (x3)
add x5 ← x2 + x7
```

- 10ns（フォワーディングありの場合）  
1,3 命令目の ld では M が終わるまで値が取れないため、  
それに依存した後ろにある命令ではバブルが生じる（ストールが発生する）



## 課題 7

- (6) 以下の命令列を実行するのに必要な時間を計算せよ  
ここで mul は乗算命令であり X に 4 サイクルが必要である。  
依存関係を満たすために必要な場合のみ  
パイプラインを適宜ストールして実行するものとせよ  
mul  $x2 \leftarrow x1 + x4$   
add  $x1 \leftarrow x2 + x3$   
add  $x5 \leftarrow x2 + x7$
- 10ns (フォワーディングありの場合)



# 課題 7

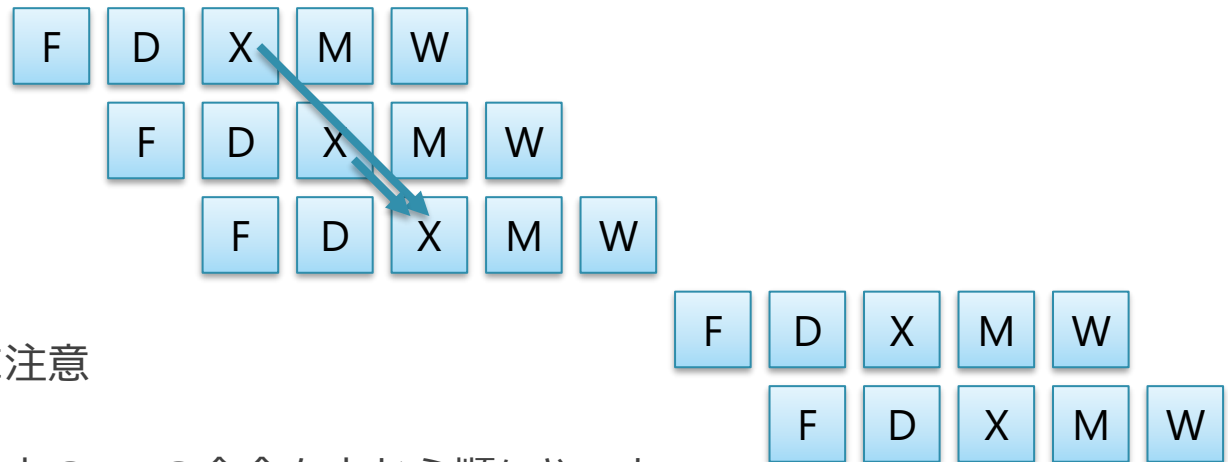
- (7) 以下の命令列を実行するのに必要な時間を計算せよ  
ここで beq はオペランドが等しい時に分岐する分岐命令である  
プロセッサは分岐予測を行うものとし, beq が分岐予測ミスを起こして LABEL に  
飛んだあとにフラッシュされてやり直した場合を想定せよ

```
li x1 ← 1  
li x2 ← 2  
beq x1 == x2, LABEL  
add x1 ← x2 + x3  
LABEL:  
add x2 ← x3 + x4
```

- 13ns  
フォワーディング  
ありの場合

やり直した後に  
結局 LABEL の下の  
命令も実行することに注意

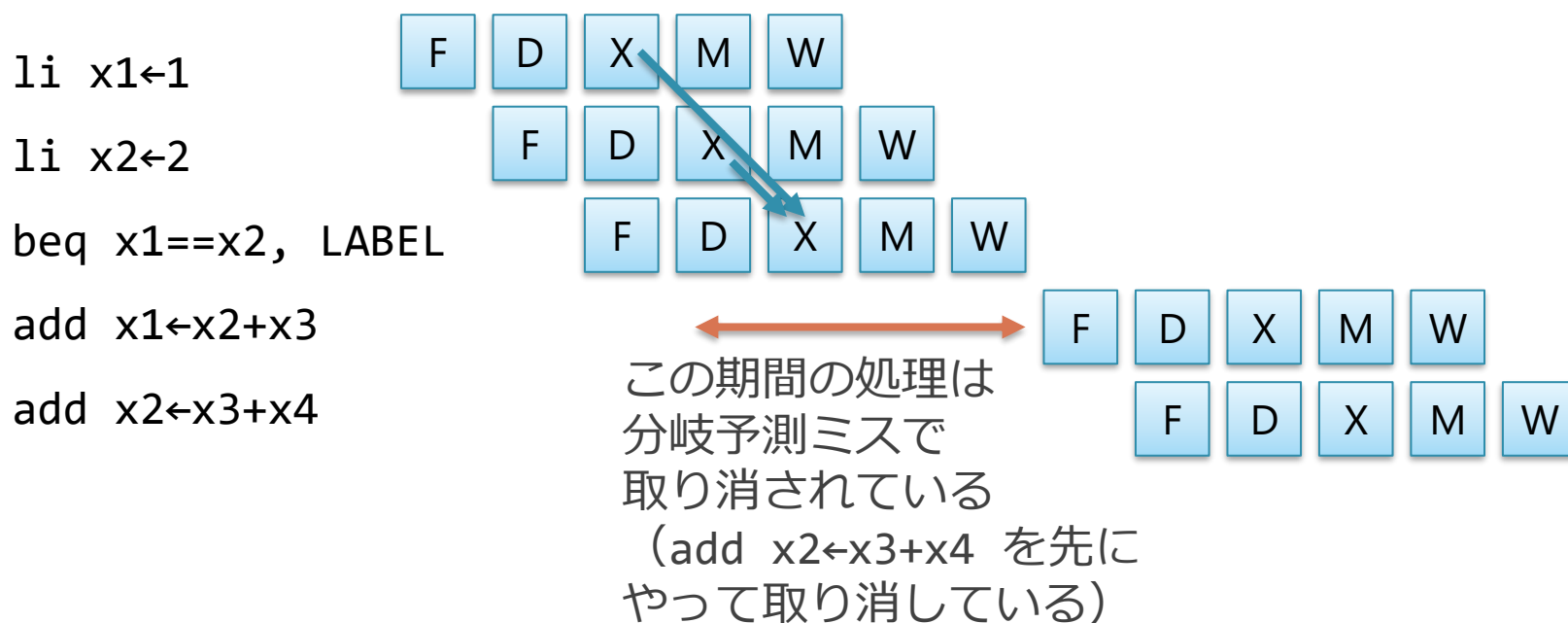
パイプラインの各行は上の5つの命令を上から順にやった  
場合に対応



# 課題 7

## ■ 13ns フォワーディングありの場合

- やり直した後に結局 LABEL の下の命令 (add x2←x3+x4) も実行することに注意



# 性能のモデル

---

- CPU の性能（処理速度） = 単位時間あたりに処理できる命令数
- 性能は以下の 2 要素のかけ算で決まる：
  1. クロック周波数 =  
1 秒あたり何サイクル分の処理ができるか
  2. Instructions per cycle (IPC) =  
1 サイクルあたり何命令処理できるか

# 性能のモデル

- 以下について検討していく
  - 理想的な場合の性能のモデル
  - ハザードを考慮した性能のモデル

# 理想的な場合の性能モデル

---

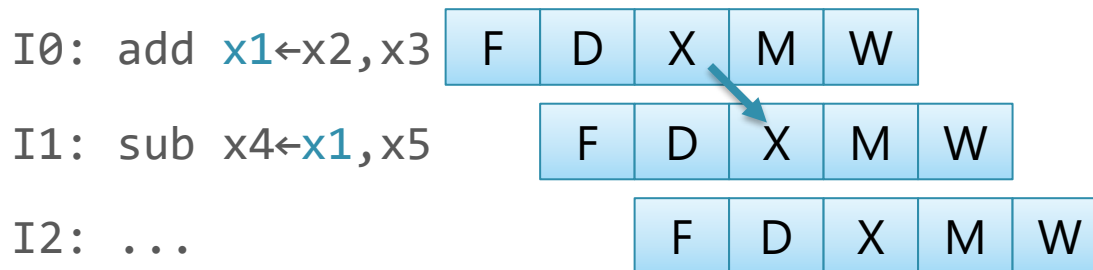
# 理想的な各モデルの性能

- 下記のそれぞれについて秒間何命令処理できるかを考える
  1. シングル・サイクル・プロセッサ
  2. スカラ・パイプライン・プロセッサ
  3. スカラ・パイプライン・プロセッサ（段数2倍）
  4. 2-way スーパースカラ・プロセッサ

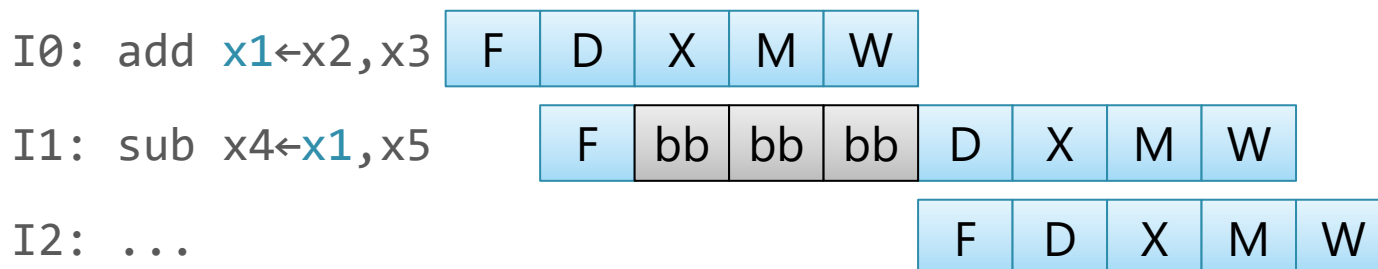
# ここからの前提： パイプラインではフォワーディングの実装を仮定

- 結果が RF に書かれる前に，後続の命令は最速のタイミングでそれを使える（=フォワーディングを行う）と想定
- つまり，依存元命令の実行ステージ（以下では X）が終わり次第，直ちに依存先の命令は実行ステージを開始できる

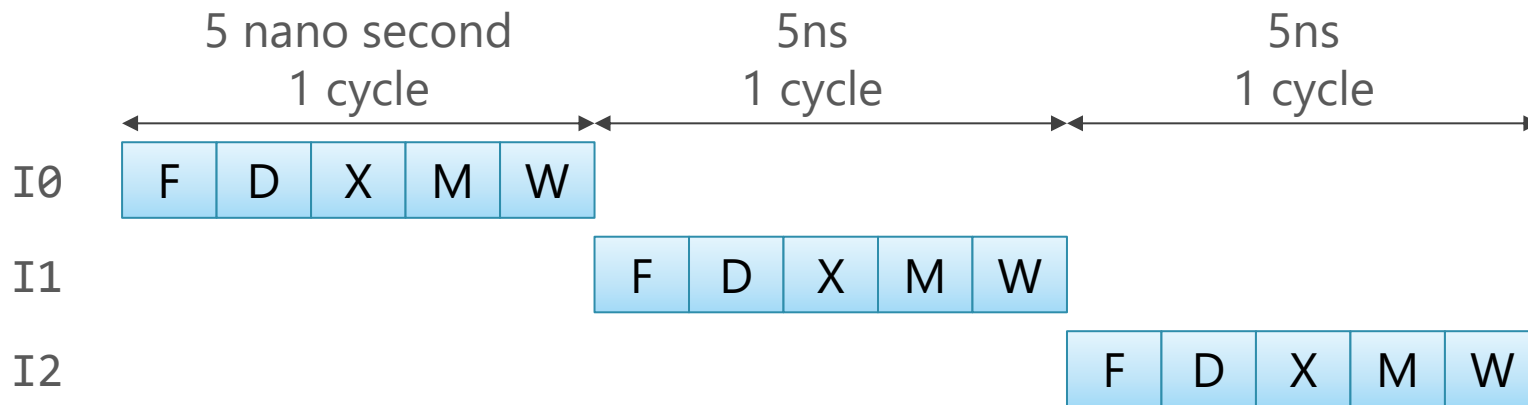
I0 の結果を I1 にフォワーディング



フォワーディングを実装しない場合，バブルが生じる

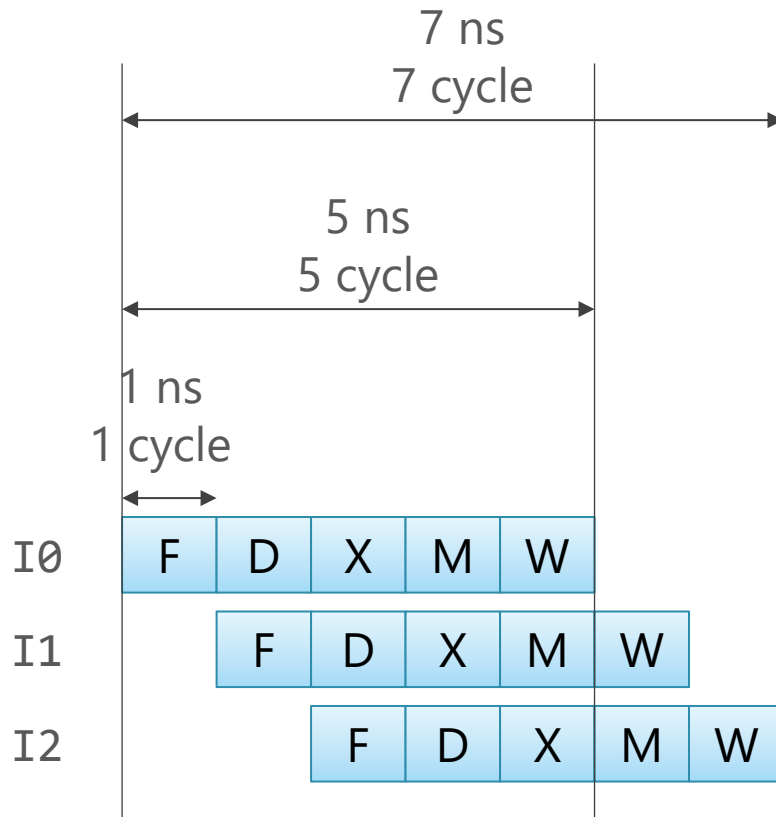


# シングル・サイクル・プロセッサ の性能



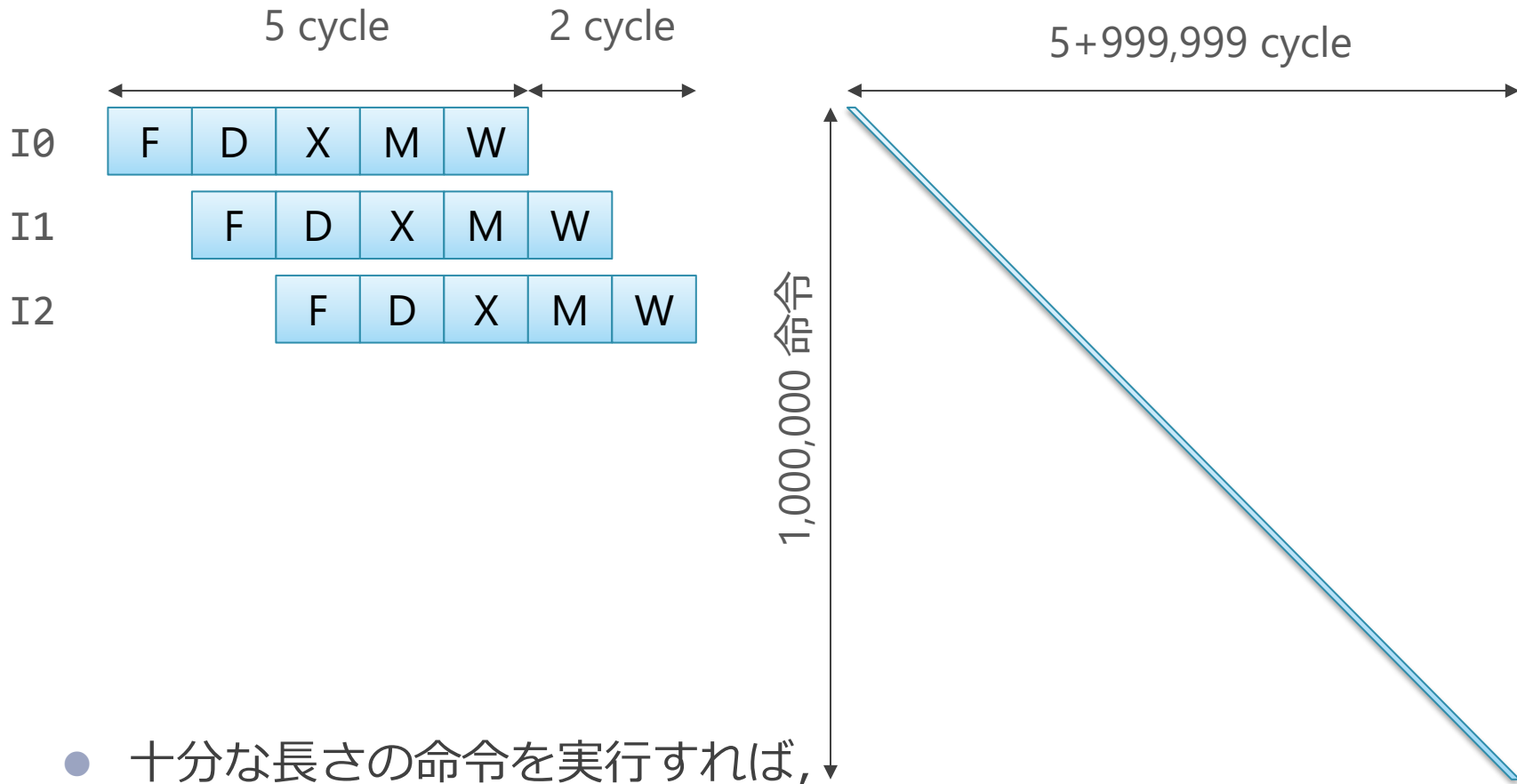
- $IPC = 1\text{命令} / 1\text{サイクル} = 1$
- 周波数 =  $1\text{秒} / 5\text{ns} = 200\text{ MHz}$  (M = メガ 100万)
- 性能 (1秒あたりの命令数) =  $IPC * \text{周波数} = \text{秒間 } 200\text{ M 命令}$

# パイプライン化されたプロセッサ の性能



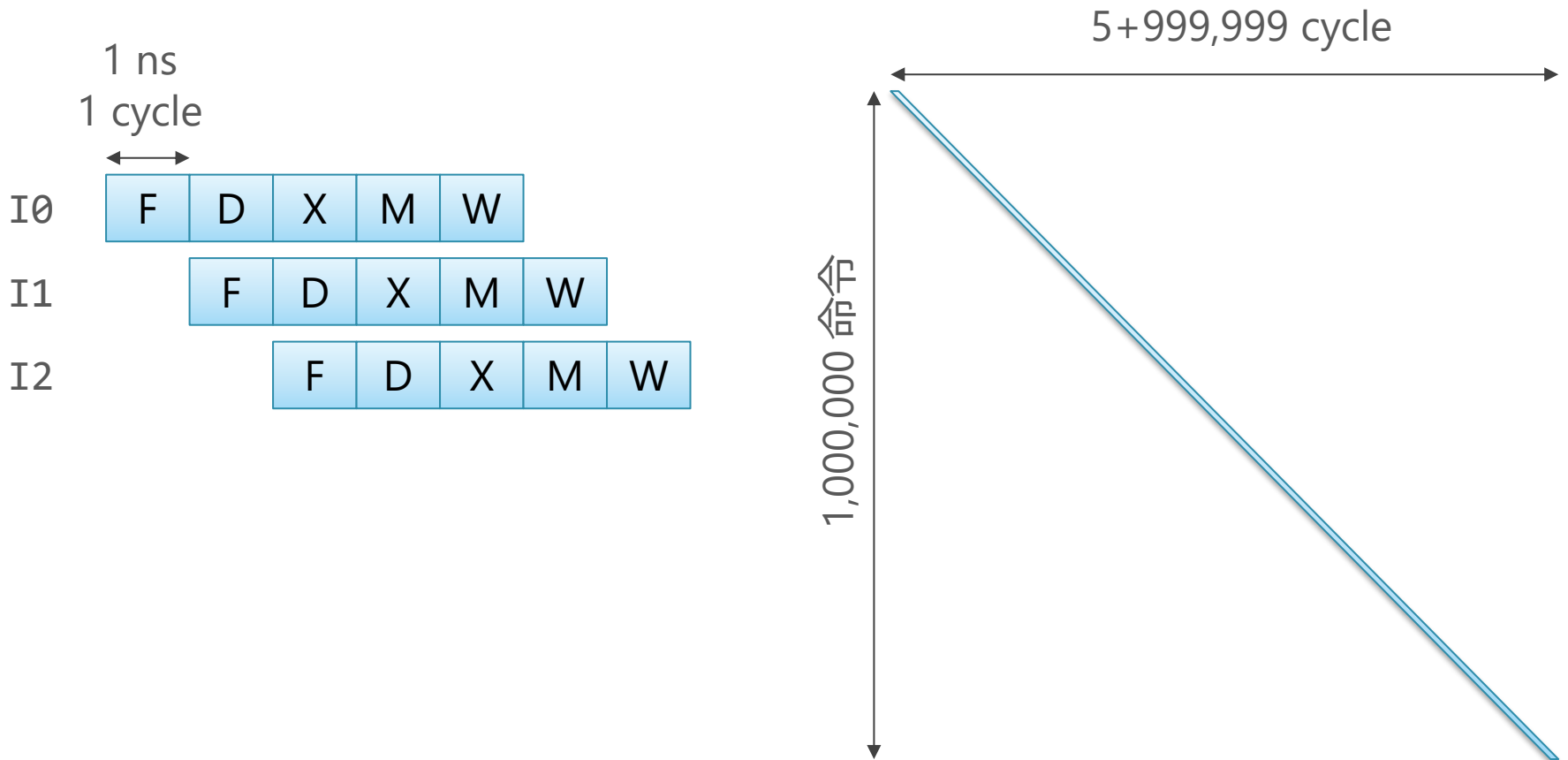
- シングル・サイクル・プロセッサの時とはサイクルの長さが変わっている事に注意
- 周波数 =  $1 \text{ 秒} / 1 \text{ ns} = 1000 \text{ MHz} = 1 \text{ GHz}$
- IPC はどう考える？ IPC = 3 命令 / 7 サイクル？

# 「十分に多く」の命令を実行した場合



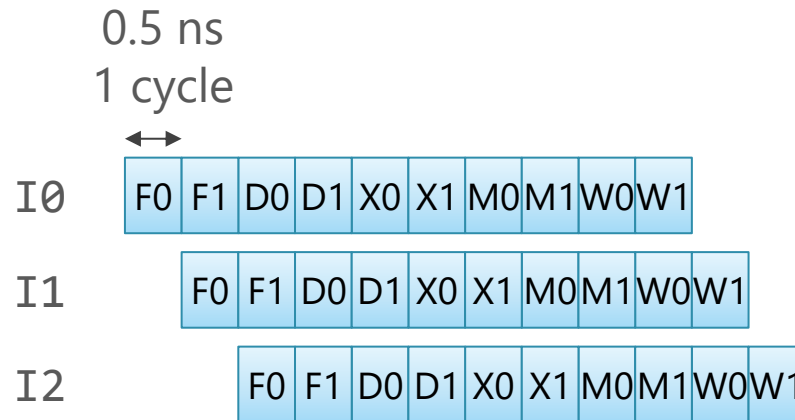
- 十分な長さの命令を実行すれば、パイプラインの長さ分は無視して近似できる
  - 右上の場合, ほぼ  $IPC = 1,000,000 / (5 + 999,999)$  は, ほぼ 1
- 左端の F ないしは右端の W ステージの傾きのみを考えればよい

# 「十分に多く」の命令を実行した場合の性能



- $IPC \approx 1 \text{ 命令} / 1 \text{ サイクル} = 1$
- 周波数 =  $1 \text{ 秒} / 1 \text{ ns} = 1000 \text{ MHz} = 1 \text{ GHz}$
- 性能（1秒あたりの命令数） =  $IPC * \text{周波数} = \text{秒間 } 1 \text{ G 命令}$

# パイプライン段数を倍にした場合の性能

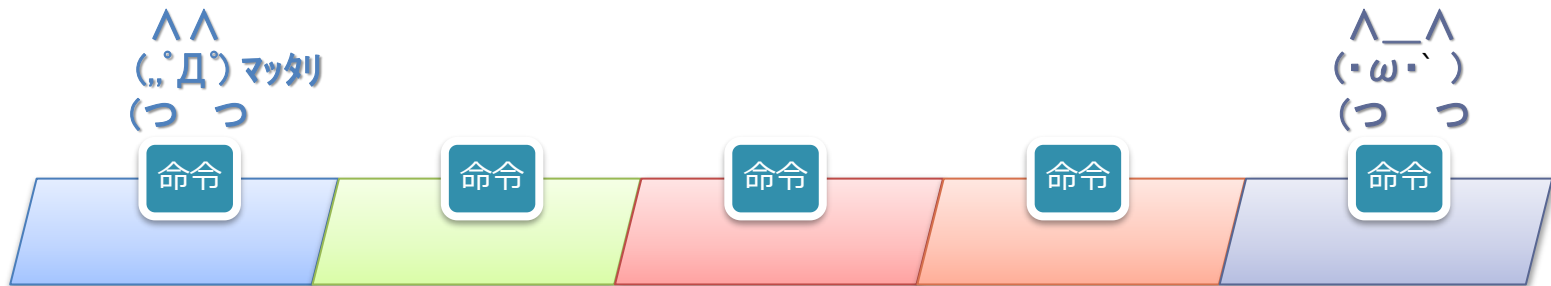


- 1 ページ前までのパイプラインのステージを半分に分割して (= 各ステージで実施する仕事を半分にして) 倍速で動かしている
- $IPC \approx 1 \text{ 命令} / 1 \text{ サイクル} = 1$
- 周波数 =  $1 \text{ 秒} / 0.5 \text{ ns} = 2000 \text{ MHz} = 2 \text{ GHz}$
- 性能 (1秒あたりの命令数) =  $IPC * \text{周波数} = \text{秒間 } 2 \text{ G 命令}$

# 各ステージで実施する仕事を半分にして、 かわりに倍速で動かしている

$IPC \approx 1 \text{ 命令} / 1 \text{ サイクル} = 1$

周波数 =  $1 \text{ 秒} / 1 \text{ ns} = 1000 \text{ MHz} = 1 \text{ GHz}$



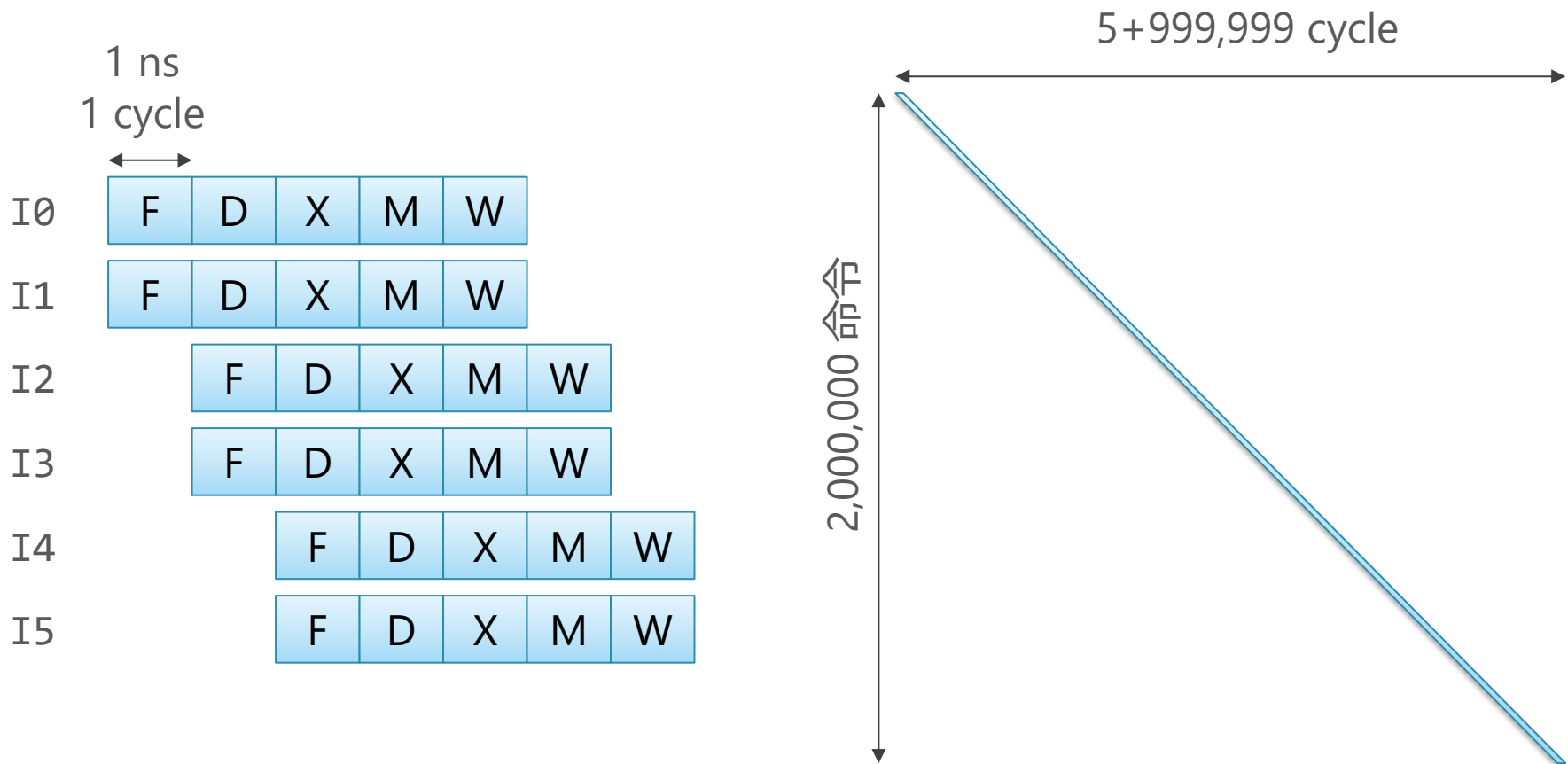
$IPC \approx 1 \text{ 命令} / 1 \text{ サイクル} = 1$

周波数 =  $1 \text{ 秒} / 0.5 \text{ ns} = 2000 \text{ MHz} = 2 \text{ GHz}$

1 人が 1 サイクルに行う仕事を半分にして、かわりに倍速で次に送る



# 2-way スーパースカラ・プロセッサの性能



- $IPC \approx 2 \text{ 命令} / 1 \text{ サイクル} = 2$
- 周波数 =  $1 \text{ 秒} / 1 \text{ ns} = 1000 \text{ MHz} = 1 \text{ GHz}$
- 性能（1秒あたりの命令数） =  $IPC * \text{周波数} = \text{秒間 } 2 \text{ G 命令}$

# ここまでの性能は理想的な性能

## ■ 理想的な各モデルの性能

- シングル・サイクル・プロセッサ : 秒間 200M 命令
- スカラ・パイプライン・プロセッサ : 秒間 1G 命令
- スカラ・パイプライン・プロセッサ (段数 2 倍) : 秒間 2 G 命令
- 2-way スーパスカラ・プロセッサ : 秒間 2 G 命令

## ■ 現実には・・・

- 回路的な要因により (第 5 回講義) ,
  - スーパスカラにするとステージごとの回路が増えて遅延が増加 → 周波数は落ちる
  - パイプライン段数を 2 倍にしても周波数は倍にならない
- 各種のハザードにより, ストールが起きて IPC が下がる (第 6 回講義)

# ハザードを考慮した性能のモデル

---

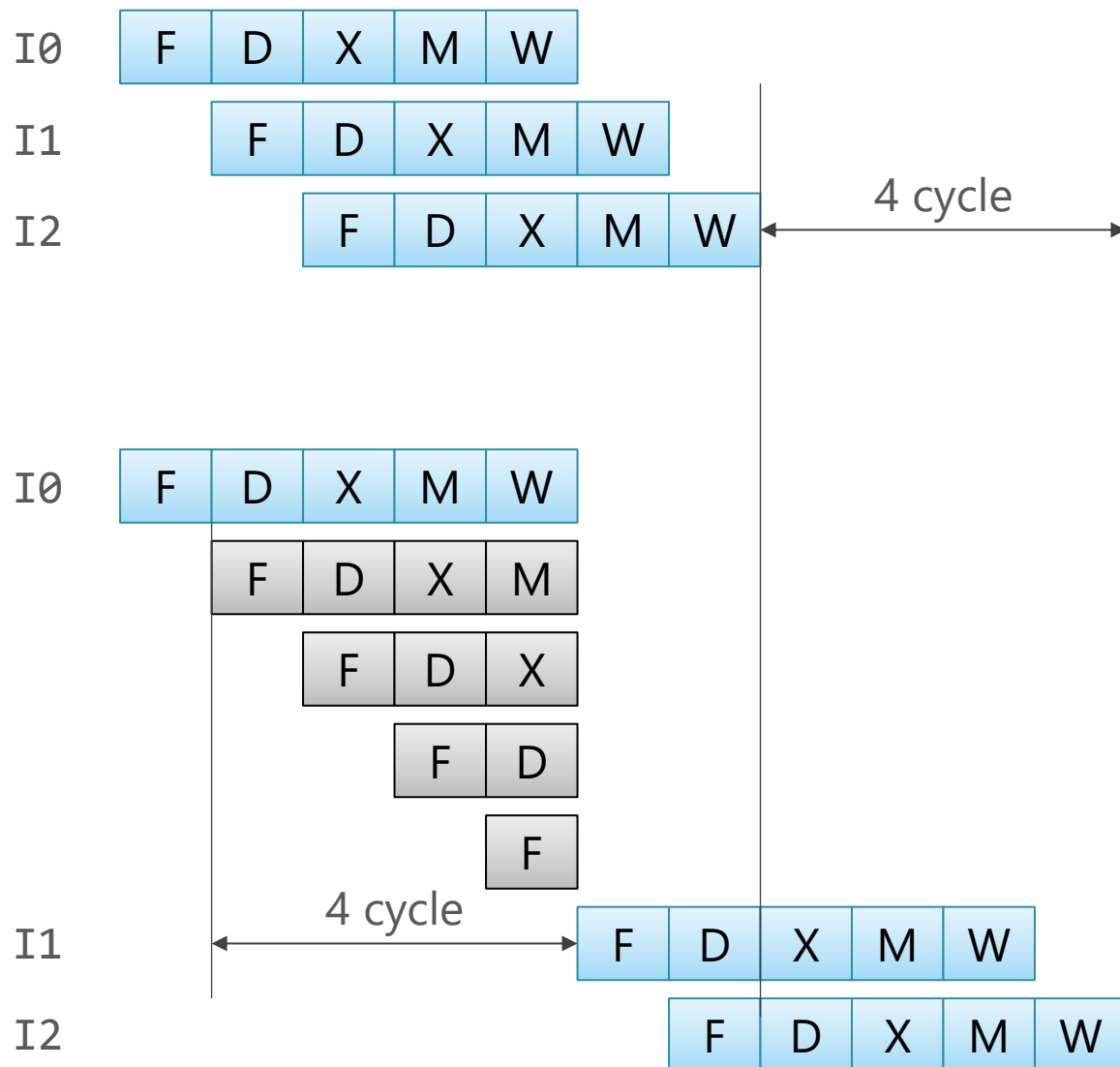
# 性能のモデル

- 以下について検討
  - 理想的な性能のモデル
  - ハザードを考慮した性能のモデル

# ハザードによる IPC の低下

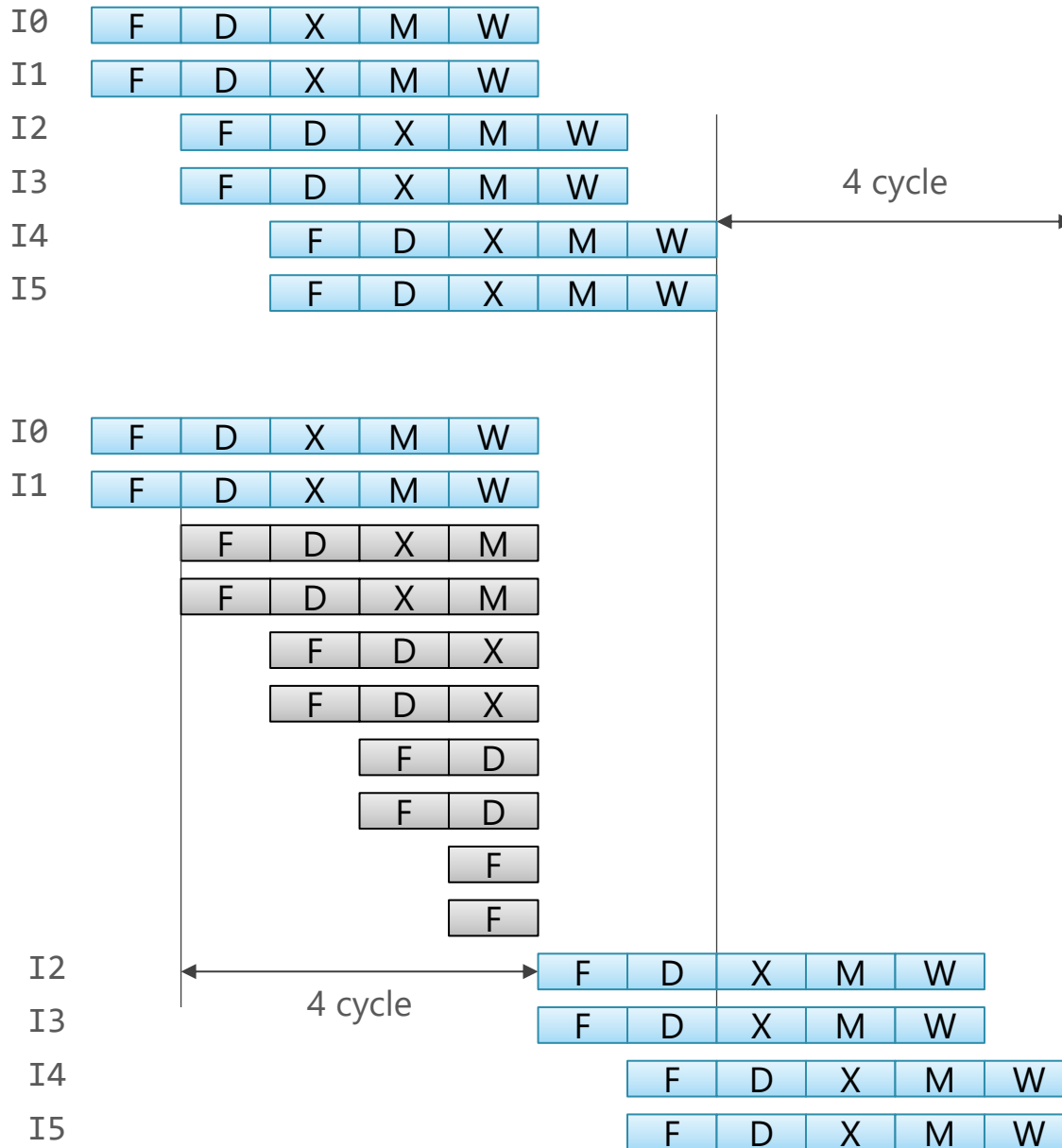
- 以下を例として, 具体的な IPC の数字の変化を説明
  - 分岐予測ミス
  - ロードに依存した命令によるデータハザード
- キャッシュによる影響も大きい
  - (この講義ではまだ説明していないが, 後日説明

# 分岐予測ミスによる実行サイクルの増加



- I2 の実行が 4 サイクル遅れている
- 4サイクル分の処理を取り消してやり直しているため

# スーパスカラの場合



- I5 の実行が 4 サイクル遅れている
- 4サイクル分の処理を取り消してやり直しているため

# 分岐予測ミスによる実行サイクルの増加のモデル

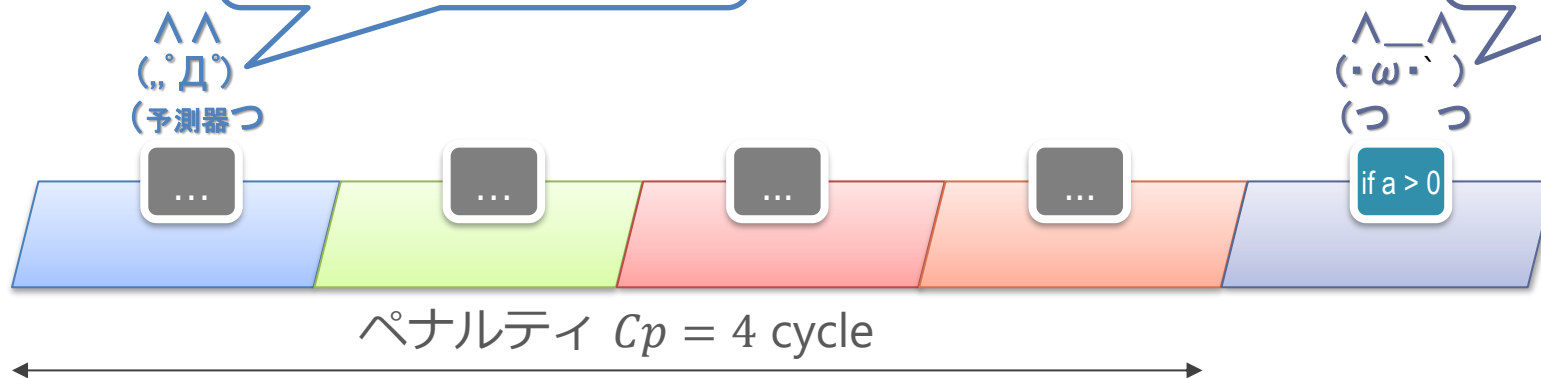
## ■ まとめると…

- 予測ミス毎に、実行サイクル数がペナルティの分だけ伸びる
  - ペナルティは（パイプライン段数 - 1）サイクル
  - スカラでもスーパスカラでも同じ

# ペナルティは（パイプライン段数 - 1）サイクル 取り消される命令数は増えるが，時間は同じ

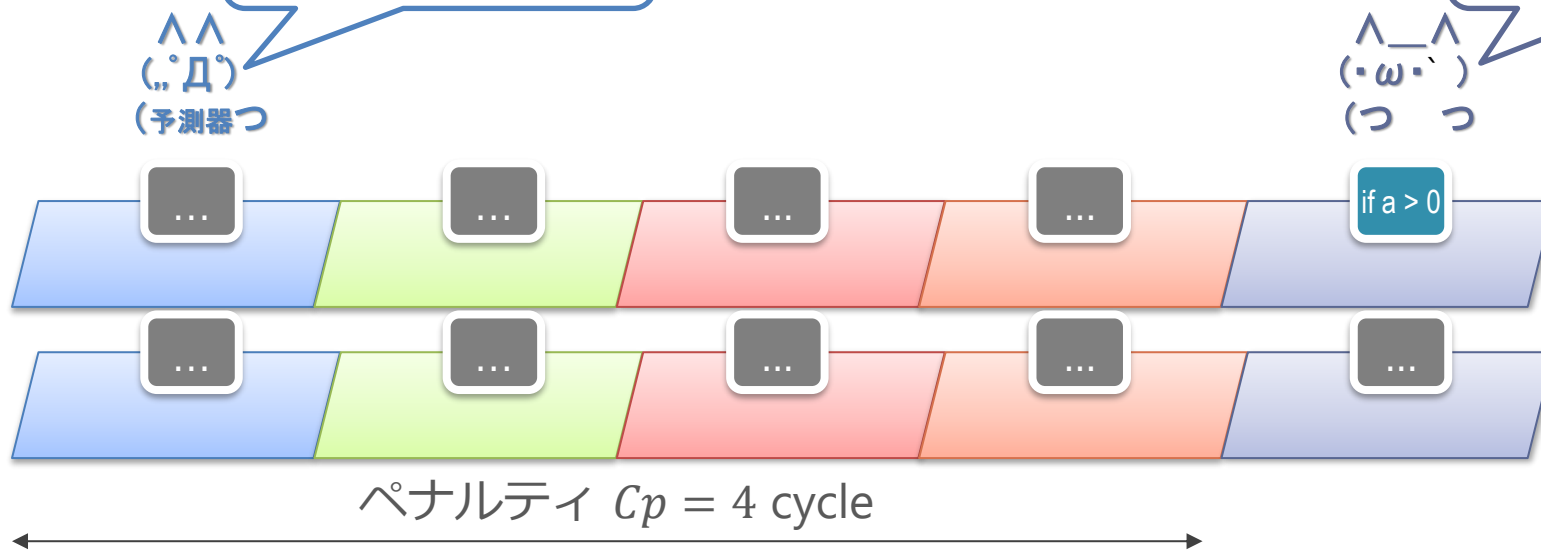
else じゃなかったかー

間違っとするがな



大損害まったなし

ライン2本分  
廃棄やん...



# 分岐予測ミスによる実行サイクルの増加のモデル

## ■ 以下のようにおいた場合,

- ミスがない理想的な実行の際のサイクル数 :  $C_t$
- 分岐予測ミスの発生回数 :  $N_m = N_i \times P_b \times P_m$ 
  - 実行命令数 :  $N_i$
  - プログラム中の分岐命令の出現率 :  $P_b$
  - 分岐命令毎の予測ミス発生率 :  $P_m$
- 分岐予測ミス・ペナルティ :  $C_p$

## ■ 実行サイクル数 $C_r$ は, 理想サイクル数 $C_t$ に対して,

- $C_r = C_t + N_m \times C_p$
- 意味 : 予測ミスの発生回数×ペナルティ だけ実行時間が伸びる

# 具体的な値を入れてみる

## ■ 以下のように置いた場合,

- 理想的な実行の際のサイクル数 :  $Ct = 1M$  (メガ)
- 分岐予測ミスの発生回数 :  $Nm = Ni \times Pb \times Pm = 0.025M$ 
  - 実行命令数 :  $Ni = 1M$
  - 分岐命令の出現率 :  $Pb = 0.25$   
(プログラムは大体 4 命令に 1 つぐらい分岐が出てくる)
  - 分岐命令毎の予測ミス発生率 :  $Pm = 0.1$
- 分岐予測ミス・ペナルティ :  $Cp = 4$

## ■ 実行サイクル数 $Cr$ は

- $Cr = Ct + Nm \times Cp = 1M + 0.1M = 1.1M$
- 分岐予測ミスにより 10% 実行サイクル数が伸びている

# IPC で考えると

- 最終的な性能を考える上で IPC の方が都合がよい

- 実行サイクル数  $C_r$  を命令数  $N_i$  で正規化すると,

- $$\frac{C_r}{N_i} = \frac{C_t}{N_i} + \frac{(N_m \times C_p)}{N_i} = \frac{C_t}{N_i} + \frac{(N_i \times P_b \times P_m \times C_p)}{N_i} = \frac{C_t}{N_i} + P_b \times P_m \times C_p$$

- IPC は命令数を実行サイクル数で割ったもの = つまり上記の逆数

- $$IPC_r = \frac{1}{\frac{C_t}{N_i} + P_b \times P_m \times C_p} = \frac{1}{\frac{1}{IPC_t} + P_b \times P_m \times C_p}$$

- ここで  $IPC_r$  は実際の IPC,  $IPC_t$  は理想 IPC

# IPC の式のまとめ

■ 
$$IPC_r = \frac{1}{\frac{1}{IPC_t} + Pb \times Pm \times Cp}$$

- 実際の IPC :  $IPC_r$
- 予測ミスがなかった場合の理想 IPC :  $IPC_t$
- プログラム中の分岐命令の出現率 :  $Pb$
- 分岐命令毎の予測ミス発生率 :  $Pm$
- 分岐予測ミス・ペナルティ（サイクル） :  $Cp$
- （実行命令数が項から消えている）

# スカラの5段パイプライン・プロセッサのIPC

- $$IPC_r = \frac{1}{\frac{1}{IPC_t} + Pb \times Pm \times Cp}$$

- ここで,  
理想 IPC       $IPC_t = 1,$   
分岐発生率     $Pb = 0.25,$   
予測ミス率     $Pm = 0.2,$   
ペナルティ     $Cp = 4$  (5段なので  $5 - 1 = 4$  サイクル) とすると,

- $IPC_r = \frac{1}{1 + 0.25 \times 0.2 \times 4} \simeq 0.83$
- つまり分岐予測ミスが無かった場合の理想 IPC と比べて  
17% ぐらい IPC が落ちている

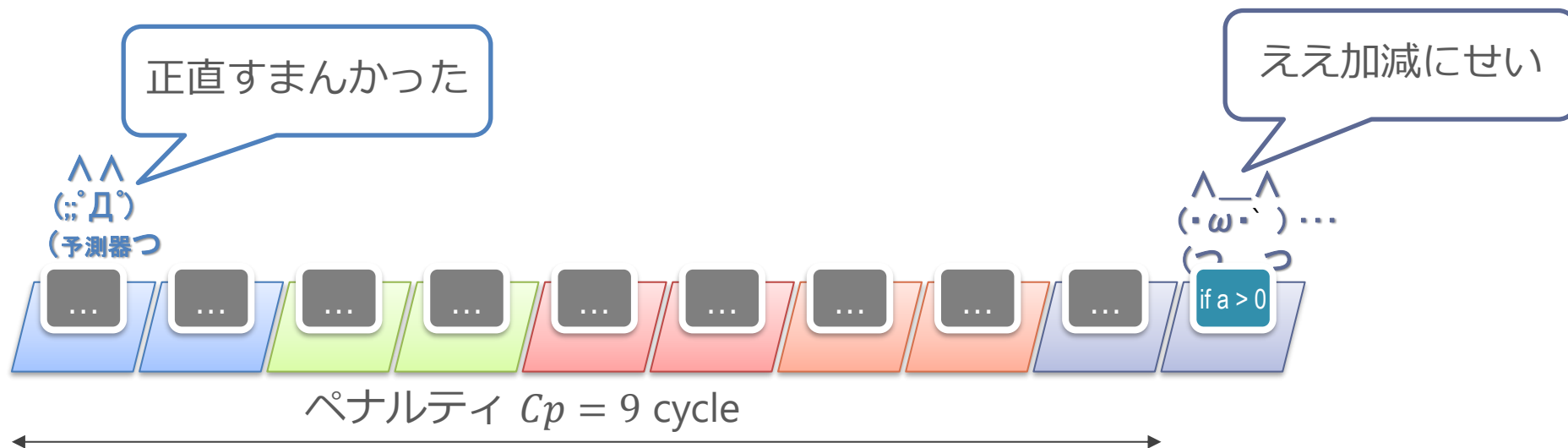
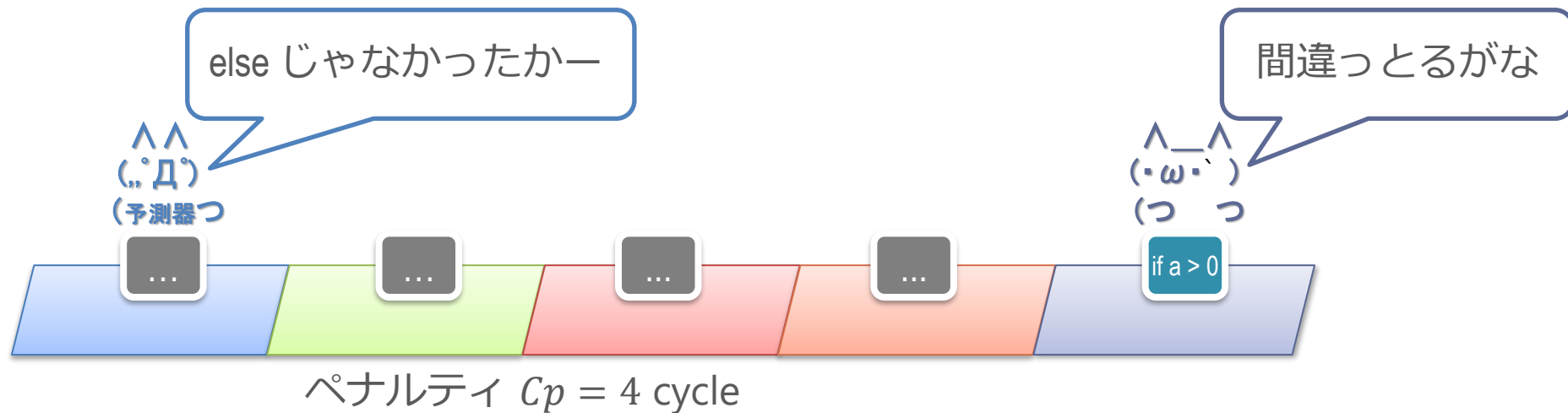
# パイプライン段数を2倍にした時のIPC

■ 
$$IPC_r = \frac{1}{\frac{1}{IPC_t} + Pb \times Pm \times Cp}$$

■ パイプライン段数を2倍にすると...

- $IPC_t = 1$ ,  $Pb = 0.25$ ,  $Pm = 0.2$  は変わらない
- ペナルティは  $Cp = 9$  (10段なので  $10 - 1 = 9$ ) に
- $IPC_r = \frac{1}{1 + 0.25 \times 0.2 \times 9} \simeq 0.69$
- 分岐予測ミスが無かった場合の理想IPC と比べて31% ぐらい性能が落ちている
  - パイプライン段数が5段の時よりIPC低下が大きい

# パイプライン段数が深い時は予測器の精度が重要



# パイプライン段数を2倍にした時の性能

- 全体の性能は周波数×IPCで決まる
  - 段数を倍にしたことで、周波数は2倍に
  - IPCは0.69
  - 性能は  $2 \times 0.69 = 1.38$

# パイプライン段数を2倍にした時のIPC

## ■ まとめると,

- 5段パイプの理想的な性能 : 1
- 分岐予測ミスありの性能 : 0.69
- 10段パイプの理想的な性能 : 2
- 10段で予測ミスあり : 1.38 (0.69 の 1.66倍)

## ■ わかること :

1. パイプライン段数を深くするとIPCへの影響が増加する
  - 同じ分岐予測ミス発生率でも, より大きくIPCが下がる
2. 予測ミス率0.2で10段パイプだと, 全体の3割ぐらいの時間は無駄な実行をしている
  - $1.38 / 2 = 0.69$

# スーパスカラにした時の IPC

■ 
$$IPC_r = \frac{1}{\frac{1}{IPC_t} + Pb \times Pm \times Cp}$$

■ スーパスカラにすると...

- $Pb = 0.25$ ,  $Pm = 0.2$ ,  $Cp = 4$  は変わらない
- パイプラインが 2 本に増えたので  $IPC_t = 2$  に

- $$IPC_r = \frac{1}{\frac{1}{2} + 0.25 \times 0.2 \times 4} \simeq 1.42$$

- 分岐予測ミスが無かった場合の理想 IPC に比べると大幅に低い

# スーパスカラにすると、その分取り消される命令が増える

else じゃなかったかー

間違っとするがな

^^  
(.°Д°)  
(予測器つ

^^  
(.ω°)  
(つ つ



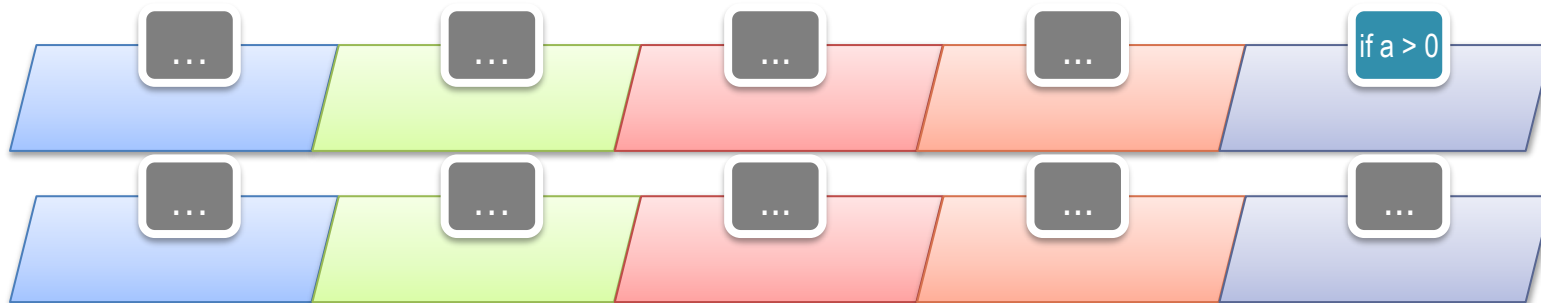
取り消される命令=4命令

大量廃棄やな・・・

また違うやん

^^  
(.°Д°)  
(予測器つ

^^  
(.ω°)  
(つ つ

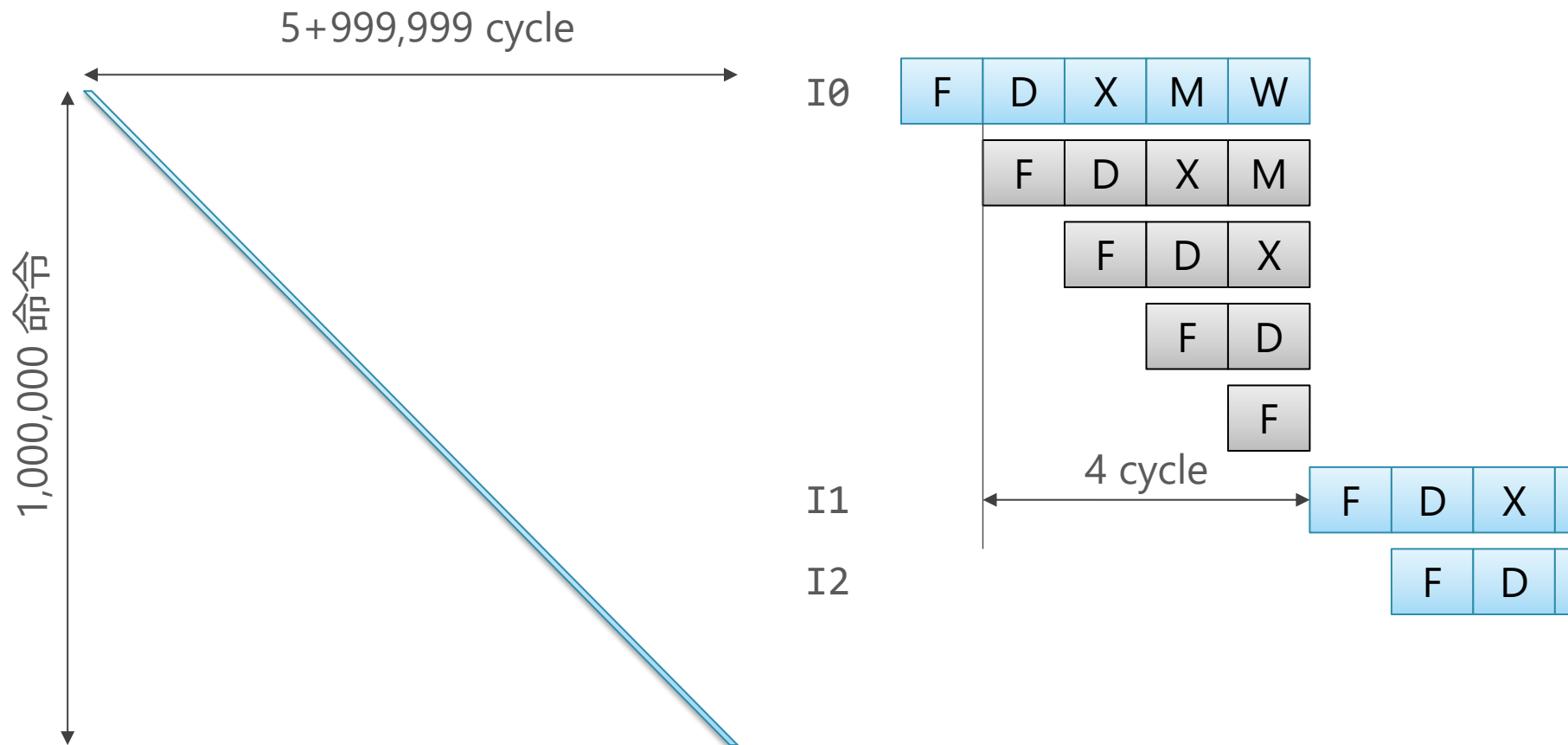


取り消される命令=9命令

# 分岐予測ミスによる IPC の低下のまとめ

- 以下のような場合に、より予測ミスの影響が大きくなる
  - パイプラインを深くする（周波数を向上させる）場合
  - スーパスカラにしてパイプライン本数を増やす場合
- 裏を返すと…
  - パイプラインを深くしたり本数を増やす回路を追加する代わりに、予測器を強化した方が性能が上がることも
  - バランスが大事
    - へちよい分岐予測器のまま他を強化しても無駄と言う事も

# パイプラインの長さと性能

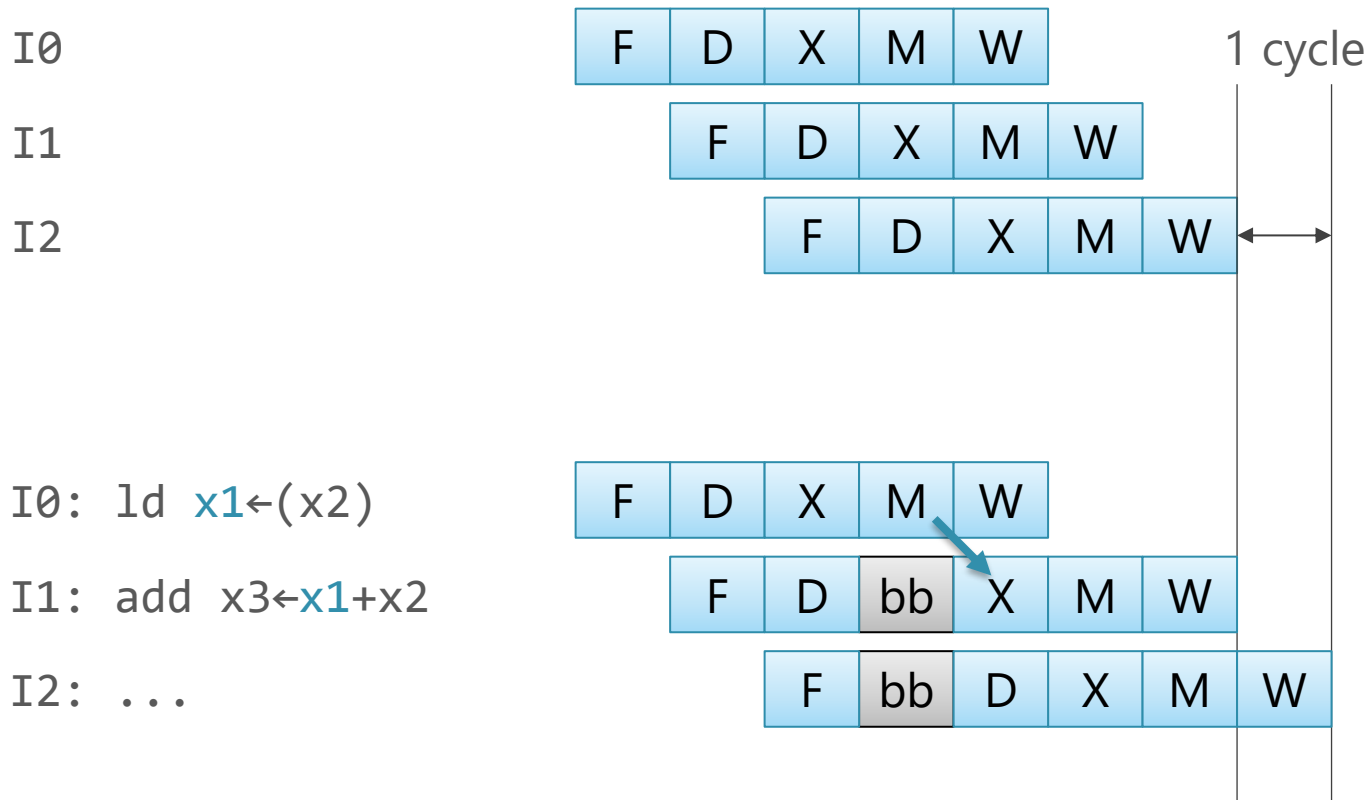


- パイプライン段数は、理想的なパイプラインではほとんど性能に影響しない
  - 最初の1命令分しか実行時間に影響しないから
- 分岐予測ミスは、起きる度にパイプラインの長さが表面化する

# ハザードによる IPC の低下

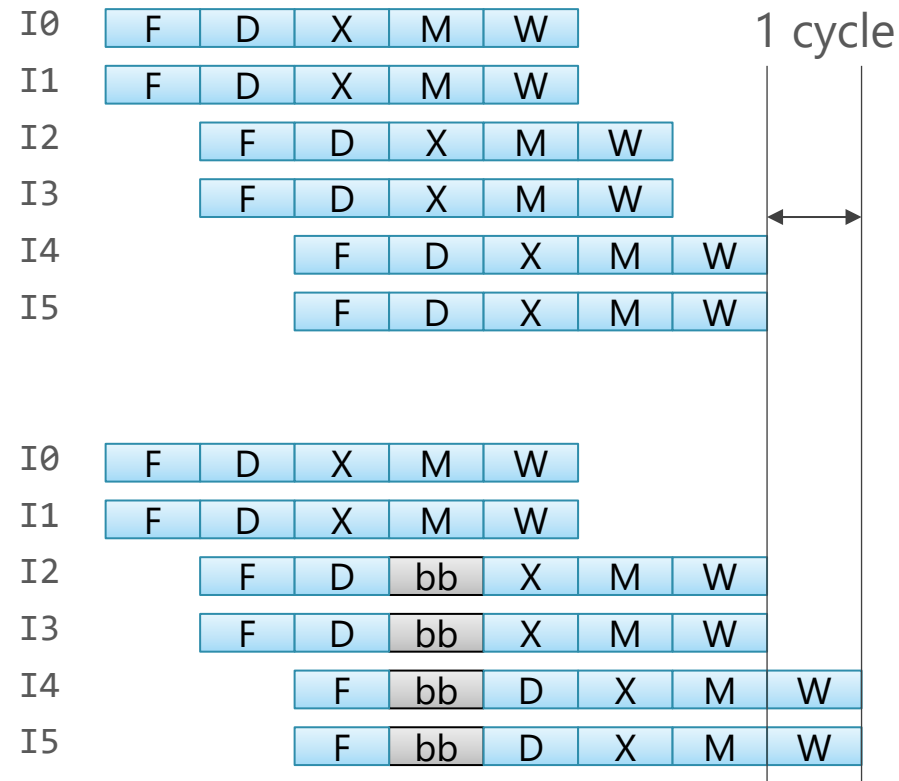
- 以下を例として, 具体的な IPC の数字の変化を説明
  - 分岐予測ミス
  - ロードに依存した命令によるデータハザード

# ロードに依存した命令によるデータハザード



- パイプライン全体が、理想的な場合と比べて1サイクル遅れている

# スーパスカラの場合も同じ



- パイプライン全体が、理想的な場合と比べて1サイクル遅れている

# ロードによるデータハザードの実行サイクルの増加のモデル

## ■ 以下のようにおいた場合,

- ミスがない理想的な実行の際のサイクル数 :  $C_t$
- ロードのデータハザードの発生回数 :  $N_m = N_i \times P_l \times P_h$ 
  - 実行命令数 :  $N_i$
  - ロードの出現率 :  $P_l$
  - ロード命令毎のハザード発生率 :  $P_h$
- ハザード時のサイクル数の増加 :  $C_p$

## ■ 実行サイクル数 $C_r$ は, 理想サイクル数 $C_t$ に対して,

- $C_r = C_t + N_m \times C_p$
- 分岐予測ミスの時とほぼ同じ

# 性能のモデルの一般化

---

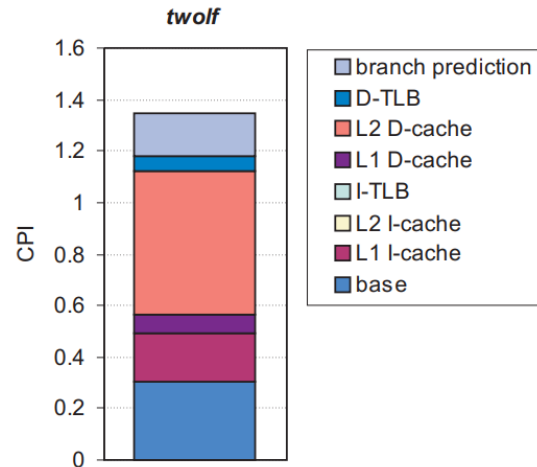
# 一般化できる

- 以下のようにおいた場合,
  - 理想的な実行の際のサイクル数 :  $C_t$
  - 何らかのハザードの発生回数 :  $N_h = N_i \times P_i \times P_h$ 
    - 実行命令数 :  $N_i$
    - ハザードを起こす命令の出現率 :  $P_i$
    - その命令毎のハザード発生率 :  $P_h$
  - ハザード時のサイクル数の増加 :  $C_p$
- 実行サイクル数  $C_r$  は, 理想サイクル数  $C_t$  に対して,
  - $C_r = C_t + N_h \times C_p$

# 一般化できる

- ハザード a, ハザード b, ハザード c ... とあった時
- 実行サイクル数  $C_r$  は, 理想サイクル数  $C_t$  に対して,
  - $C_r = C_t + N_{ma} \times C_{pa} + N_{mb} \times C_{pb} + N_{mc} \times C_{pc} + \dots$
  - これを積み上げ棒グラフで表したのが CPI Stack

# CPI Stack



**Figure 1.** Example CPI stack for the *twolf* benchmark.

- Cycles Per Instruction (CPI) は 1 命令あたりのサイクル数
  - $Cr = Ct + Nma \times Cpa + Nmb \times Cpb + Nmc \times Cpc + \dots$  を命令数で正規化して図示
  - $Ct = 0.25$  ぐらいなので, 4 way スーパスカラが基準?
- どのようなハザードによって性能が決まっているのかを直感的に知ることができる

# Out-of-order スーパースカラ・プロセッサの場合

- Out-of-order スーパースカラ・プロセッサではもっと複雑
  - 長時間かかる命令による待ちがあった場合, それを待つ間に別の後ろにある命令を実行できる
  - 色々なハザードが同時並行で起きうる
- 単純な線形加算のモデルでは扱えない
  - $Cr = Ct + Nma \times Cpa + Nmb \times Cpb + Nmc \times Cpc + \dots$
  - これはハザード発生時に CPU 全体がストールするからなり立つ
  - ハザード同士がオーバラップして起きると上手く扱えない
- 色々な方法が研究されている

# ここまでのまとめ

## ■ 性能は以下の 2 要素のかけ算で決まる：

1. クロック周波数
2. Instructions per cycle (IPC)

## ■ 色んなバランスが大事

- クロック周波数だけを上げてても IPC が上がらないとだめ
- スーパスカラの同時実行数を増やしても IPC が増えない事もある
- 予測器だけ強くしても IPC があまり上がらないことも

# 実際には構成をどう決めるのか？

- だいたい最初に以下あたりの条件が決まっている
  - 要求性能
  - 使える回路面積（チップの大きさ）
  - 許容出来る消費電力

# いろんな戦略がありえる

- 回路面積が一定の場合、どちらが良いか？
  - パイプラインの本数を増やす
  - 予測器を複雑にして精度を上げる
- 消費電力の上限が一定の場合
  - 消費電力はクロック周波数の2～3乗で増える
  - クロック周波数をあえて下げるかわりに、パイプライン本数を増やしてIPCを上げる
- 色々シミュレーションやモデル化して決める

# まとめ

- 性能は以下の 2 要素のかけ算で決まる：
  1. クロック周波数
  2. Instructions per cycle (IPC)
- 以下について検討
  - 理想的な場合の性能のモデル
  - ハザードを考慮した性能のモデル
- 性能は色々な要素の相互作用で決まる

## 課題 8

### ■ 以下のような条件を考える

- 10段のパイプラインを持つ 2-way スーパースカラプロセッサであり, 理想的には  $IPC=2$  で実行できる
- 全実行命令におけるなんらかのデータハザードの発生率は 0.2
- このデータハザード発生時は 1 サイクル実行時間が伸びるものとする
- 全実行命令における分岐命令の出現率は 0.2
- 分岐予測ミス率は 0.3

- ### ■ (1) この CPU を改良する際,
- 「3-way スーパースカラにする」
  - 「2-way のまま15段パイプラインにする」
  - 「2-way のまま分岐予測器を改良する」

のどれが最も性能が上がるかを性能を計算して検討せよ

- この CPU を 3-way スーパースカラにすると理想的には  $IPC=3$  で実行できるがデータハザードの発生率は 0.3 に上昇するとする
- また, 分岐予測器を改良すると分岐予測ミス率が 0.2 にまで削減されるとする

## 課題 8

- (2) ここで効率を表す「性能エネルギー比（性能 / 消費エネルギー）」という指標を導入する。この数字が大きいほど小さなエネルギーで速く動くことを意味する。前述の3つの改良方針のうち、以下の追加の条件のもとで、最も性能エネルギー比が良いものはどれかを計算して検討せよ。
  - クロック周波数を  $N$  倍にすると、消費エネルギーは  $N^2$  倍増える
  - 3-way スーパースカラにすると消費エネルギーは 1.5 倍になる
  - 分岐予測器を改良すると消費エネルギーは 1.1 倍になる

# 提出方法

## ■ 以下を提出：

### 1. 課題 8：

- 提出は Moodle の「課題 8」のところからお願いします
- 紙に書いた場合は写真を撮ってアップロードしてください

### 2. 感想や質問：

- 「感想や質問」のところに投稿してください
- わからない場所がある場合、具体的に書いてもらえると良いです

## ■ 提出締め切り

- 次回講義開始まで

## ■ 注意：

- 課題の出来は、ある程度努力したあとがあれば良しです
  - 必ずしも正解していなくても良いです

# 休講と期末試験について

- 7/1 の講義はおやすみです
  - 次回は 7/8
- 期末試験は 8/5 に実施予定です

# 質問とか感想

---

# 質問とか感想

- 最近暑くて夏バテ気味です。先生は夏バテ対策は何かありますか？

- ストールを用いるときと、フォワーディングや遅延を用いるときの違いがよくわからないのですが、ストールを用いるのは、依存関係がldとaddの時のみですか？

- デコードのタイミングが分らないです、あと演算は同時に処理されることはないのですか、 、 ？

- また、個人的に火曜日が1、4限だけなのでこの授業をZOOMでも配信してもらえるととても嬉しいです。先生も暑い中自転車でくるのは大変だと思うので、、、ぜひご検討よろしくお願いします。

- 課題について、フォーワーディングをするとどのステージまでストールしたら良いのか分かりませんでした。

- 前回の内容と今回の内容とのつながりがイマイチわかりませんでした。パイプライン化の話はなんとなくわかったのですが他があんまりわからなかったです。一応課題は出しているのですが完全に理解しているのではなく、多分こうだろうみたいな感じなので期末テストが心配です。

# 質問とか感想

- 今日の課題が難しかったです、、。課題に関してですが、beq が分岐予測ミスを起こすのはどのような場合ですか？

- 分岐予測が正しかったかが分かるのはその命令のメモリアクセスの処理が完了した時という認識であっていますか？

- 最大周波数についてよく分かりませんでした。授業資料を読み返してみましたが、見つけられなかったのもう少し教えて欲しいです。

# 質問とか感想

- フォワーディングを行なっているときのロードについての質問です。第6回p64で、MEMより後にEXが来ないといけないとありましたが、理由があまりわかりませんでした。ldのMEMを行った時点で初めてロード結果が取得できるというのはわかるのですが、ldのMEM実行後にMEMが取得してきた結果を手元に置いていたとしても、その後でldの結果を用いる命令addのEXを行ったところで、addのEXはaddのMEMより前の段階の処理なので、MEMが手元に置いている値は使えないのではないかと思います。

# 質問とか感想

- 課題の(5)や(6)でどのタイミングでbbを入れれば良いのかがよくわかりませんでした。例えば(6)の場合、3行目の処理はF D bb bb bb X M W と F bb bb bb D X M W のどちらが良いのでしょうか。また(5)で、ファイル提出の方で添付した画像の3つの解答だと、どれが正しくてどれが間違っているというのはあるのでしょうか。

- 以下の命令列を実行するのに必要な時間を計算せよ、のような問題で、もし命令が、「①ld命令、②ld結果を使うadd命令、③ld結果を使わないadd命令」のように並んでいた場合、処理手順を並べ替えて①③②の順で実行したものが解答になるのでしょうか。

# 質問とか感想

- とても前の回の質問で申し訳ないのですが、課題4の解答で、`li 0→B` とレジスタBに0を入れて、最後のbでの比較の値に使っていますが、`sub A, 1→A`で使っている1は、なぜ`li 1→C`のようにしないで即値でやって良いのでしょうか。

- また、そこまで授業内容とは関係ないのですが、「PCが落ちる」というのは具体的に何がどうなったときに起こるのか知りたいです。

- 課題の状況のように、フォーワーディングが行われる際、ストールがどこに必要でどこに必要でないかがよくわからなかった。

- [11:0]や[20|10:1|11|19:12]などの部分が色々種類がありよく分かりませんでした。授業終了後は次の予定があり質問に伺えなかったのでご教示いただければ幸いです。

- 63ページでMEMがないのはなぜですか？

- 課題(7)について、誤った処理がフラッシュされるまで正しい方のフェッチとデコードの処理は行われないと考えて良いのでしょうか？

- データ依存には偽の依存があり、偽の依存はさらに逆依存と出力依存の2つに分けられるという話がありました。逆依存と出力依存の解決方法として、使用するレジスタが被らないようプログラムを書き換えるという方法が講義内で紹介されていましたが、同じ方法で解決できるならそこまで厳密に区別しなくてもいい(=まとめて偽の依存と扱ってもいい)のでしょうか。それとも、依存が発生した原因に基づいて分類しておいた方が良いでしょう。

- また、今回の講義の内容と直接的には関係ないのですが、C言語のポインタという概念がどのようなものなのか簡単に教えていただきたいです。

- 課題について、LABELに飛んだ後、どのサイクルでフラッシュが起こりやり直すのかがわからなかった。

- 最近の授業についていけてなくて、なぜ理解しにくいのが分からないです。

- 課題7でストールを実行するときいつまでバブル状態でいけばよいかを考えるのが難しかったです。とくに、Mが終わるまでなのか、Xが終わるまでなのかを考えるのに苦戦しました。パイプライン・ステージを一文字として紙のノートに絵を描くと確かに考えやすかったです。まだ全然慣れることができていなくて時間がかかってしまうのではやく慣れるように練習を重ねていこうと思います。

- Out-of-Order実行におけるフロントエンド／バックエンドという用語は、Web開発におけるフロントエンド／バックエンドとは異なる文脈で用いられているということは今回初めて知りました。

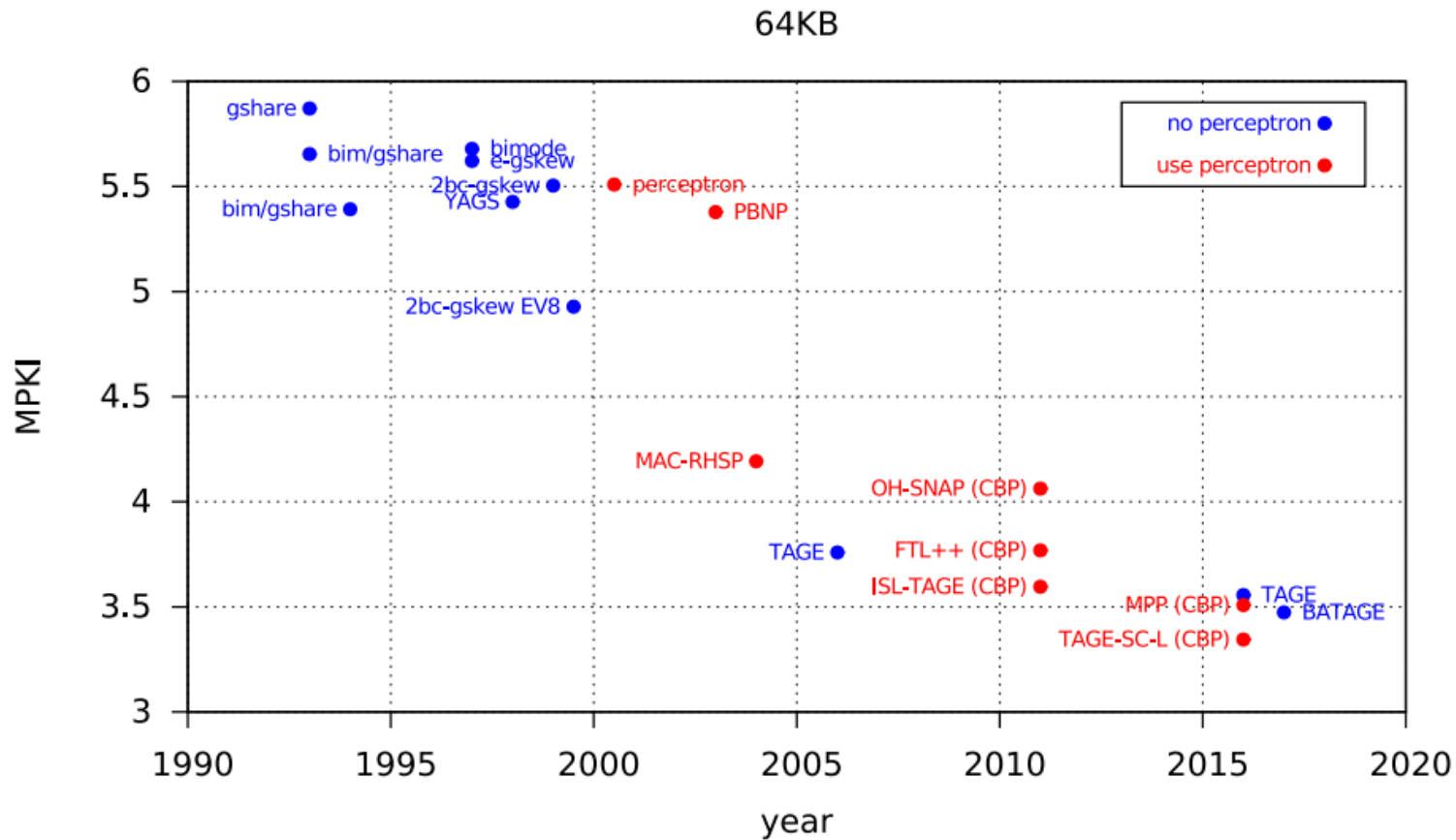
- 分岐予測について、最低どれくらいの精度があれば実用に耐えうるとされていますか？また分岐予測のコンテストで勝つレベルのシステムになるとどの程度の的中率になるのか気になりました。

# 質問とか感想

- また、分岐予測のアルゴリズムについて具体的に教えていただきたいです。特に、どのような種類があり、それぞれの精度の違いはどのようなものなののでしょうか。また、これらのアルゴリズムが実際にどのように実装され、どのように動作するのかについても詳しく知りたいです。
- 院生むけの講義資料ですが、興味があればこちらも
- <https://github.com/shioyadan/advanced-computer-organization/blob/master/aco-shioya-06.pdf>
- <https://github.com/shioyadan/advanced-computer-organization/blob/master/aco-shioya-appendix-bpred.pdf>

# 現代の分岐予測器の性能（64 KB 使用の場合）

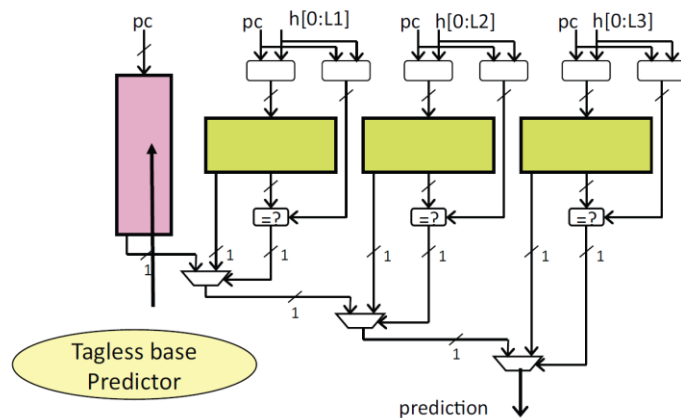
PIERRE MICHAUD, An Alternative TAGE-like Conditional Branch Predictor, TACO 2018 より



おまけ

---

# 分岐予測のアルゴリズム



## ■ André Seznec 先生が作った TAGE という予測器が 現在最も予測精度が高いと言われている

- A. Seznec and P. Michaud. A case for (partially)-tagged geometric history length predictors, *Journal of Instruction Level Parallelism* (<http://www.jilp.org/vol8>), 2006
- 写真 は <https://team.inria.fr/pacap/members/andre-seznec/> より

## ■ Championship Branch Prediction という大会がある

- 分岐予測は, この大会で出た様々なアイデアを元に発展してきた
- Seznec 先生がもうずっと優勝し続けてる (20年以上)
- 今年は久々に開催されるので, 塩谷も挑戦



# CBP2025 Opening Remarks

Sixth Championship Branch Prediction (CBP)

Co-located with ISCA 2025 – Tokyo, Japan

Rami Sheikh  
Organizing Committee Chair

June 21, 2025

arm

# 優勝！

## CBP2025 Winners ...

Nagoya Institute of Technology  
The University of Tokyo

CBP2025 Winner



1st Place \$ prize: \$1,000

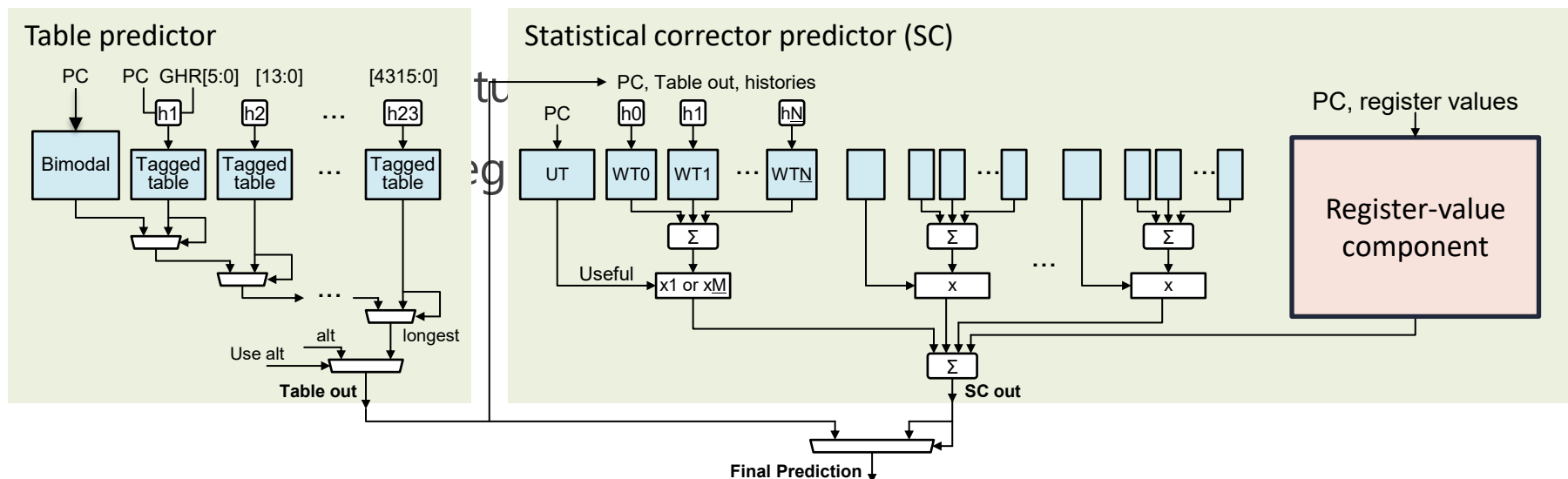


arm

# RUNLTS Predictor

我々の世界最強の分岐予測器（2025年6月現在）

- 基本的には TAGE 予測器の構造を踏襲
- レジスタの値と分岐の方向の相関を拾う方式を新しく提案

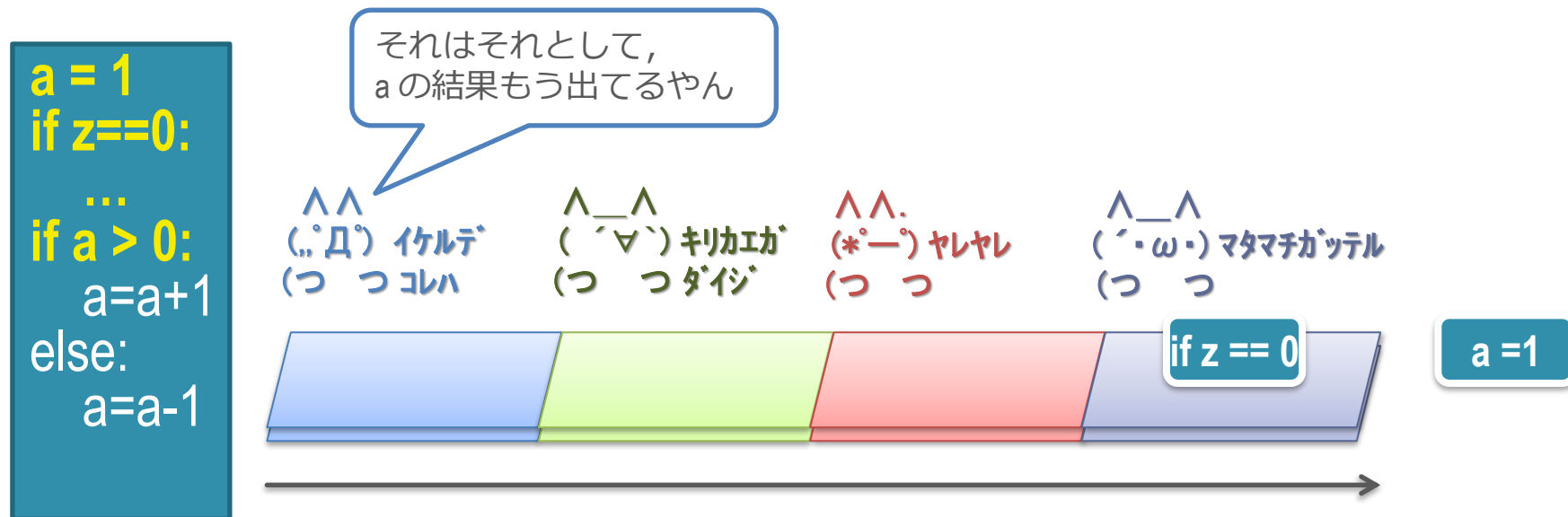


# 分岐命令の処理と制御ハザード



- 「if a > 0」の結果は最終段の(´・ω・`)の人まで反映出来ない
  - 先頭は次に a=a+1 と a=a-1 のどちらを取り込めばいいのかわからない

# RUNLTS のやりかた



- 連続した分岐の1個目（「if z==0」）でミスがあった場合を考える
  - 「if z==0」の後には一旦取り消される
  - このときしかし、その前に決定された値「a=1」が観測可能
    - これを使う